

OOP MASTERY

MODELING THE DIGITAL WORLD THROUGH OBJECTS

CORE PARADIGM

Transitioning from procedures to entities.

| CLASSES & OBJECTS

The Blueprint

A Class is a template or blueprint that defines the variables and methods common to all objects of a certain kind. It represents the concept.

The Instance

An Object is a concrete instance of a class. It carries actual data and occupies memory space, interacting with other objects.

THE FOUR PILLARS

The fundamental methods that define Object-Orientation.

| 1. ENCAPSULATION



Data Hiding

Bundling data and methods into a single unit (class) while restricting direct access to internal state.



Protection

Using access modifiers (Private, Public) to protect integrity and prevent unauthorized external interference.



Modularity

Each object is self-contained, making code easier to maintain, troubleshoot, and update independently.

| 2. ABSTRACTION

Simplify Complexity

Abstraction is the process of hiding internal details and exposing only the essential features of an object.

Think of a car dashboard: you see the speedometer and fuel gauge (the interface) without needing to understand the engine's combustion cycles (implementation).



| 3. INHERITANCE

Code Reusability

Allows a new class (subclass) to inherit attributes and behaviors from an existing class (superclass), promoting hierarchical structure.

Logical Extension

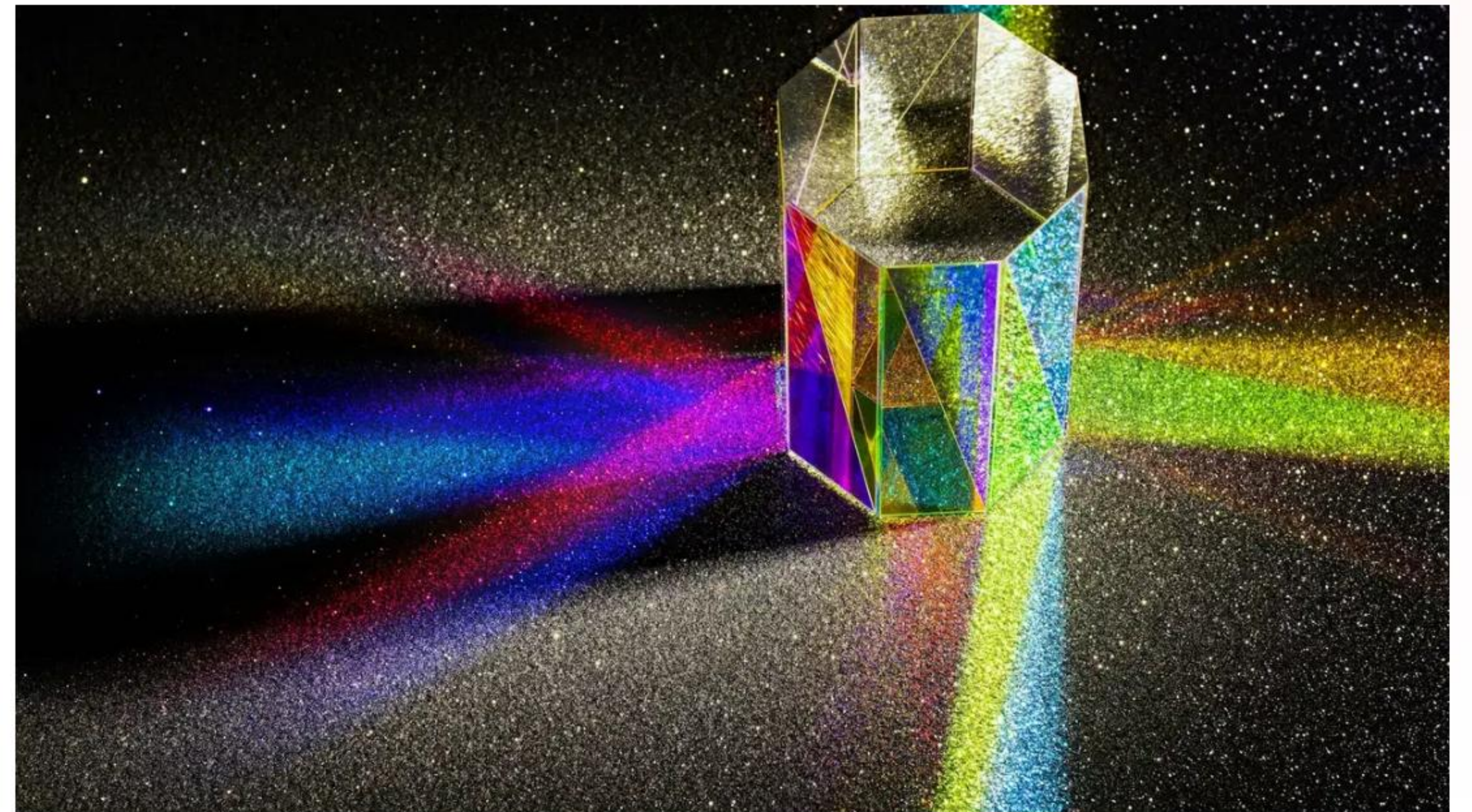
Subclasses can override or extend inherited methods, allowing developers to build specialized versions of general concepts efficiently.

| 4. POLYMORPHISM

Multiple Forms

Polymorphism allows objects of different types to be treated as instances of a common superclass, enabling flexible interfaces.

The same method call (e.g., `.draw()`) can behave differently depending on the object it is called upon (e.g., Circle vs. Square).



| IMPACT ON SYSTEMS

60%

REDUCTION IN CODE REDUNDANCY

Scaling with Confidence

Through inheritance and polymorphism, development teams can reduce boilerplate code, leading to faster development cycles and fewer logic errors across large-scale software systems.

PARADIGM COMPARISON

Feature	Procedural Programming	Object-Oriented (OOP)
Core Focus	Functions and Logic	Data and Entities
Structure	Top-Down Approach	Bottom-Up Approach
Data Access	Global/Unrestricted	Restricted via Encapsulation
Reusability	Limited to Functions	Classes and Inheritance

| REAL-WORLD APPLICATIONS

- ❑ Banking Systems: Encapsulating balances and account security protocols.
- ❑ Game Development: Inheriting character traits and polymorphic ability behaviors.
- ❑ E-commerce: Abstracting payment gateway logic while ensuring data privacy.
- ❑ Simulation: Modeling complex real-world physical systems through modular objects.

QUESTIONS?

THANK YOU FOR YOUR ATTENTION

www.code-architecture.com | contact@oop-paradigm.io

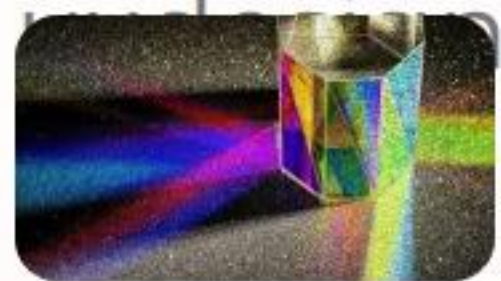
| Image Sources



https://miro.medium.com/v2/resize:fit:2000/1*Nsp3rQHcjyM1hbZllyX5Uw.jpeg

Thumbnail Source: uxdesign.cc

for



<https://media2.dev.to/dynamic/image/width=1280,height=720,fit=cover,gravity=auto,format=auto/https%3A%2F%2Fdev-to-uploads.s3.amazonaws.com%2Fuploads%2Farticles%2Fpjmfmzqqksmn824w5ge2l.jpg>

Source: dev.to